

编 号	题 号	标 题	知 识 点	代 码 作 者
例 2-1	POJ 2388	Who's in the Middle	快速选择问题	陈锋
例 2-2	UVa 1121	Subsequence	线性扫描：前缀和；单调性	刘汝佳
例 2-3	UVa 11078	Open Credit System	扫描：维护最大值	陈锋
例 2-4	UVa 11549	Calculator Conundrum	Floyd 判圈	刘汝佳
例 2-5	UVa 1152	4 Values Whose Sum is Zero	中途相遇	陈锋
例 2-6	UVa 11572	Unique Snowflakes	滑动窗口	刘汝佳
例 2-7	POJ 2823	Sliding Window	滑动窗口：单调队列	陈锋
例 2-8	UVa 1398	Meteor	线性扫描：事件点处理	刘汝佳
例 2-9	POJ 1804	Brainman	逆序对数计算	陈锋

2.2 贪心算法

贪心算法是一种解决问题的策略。如果策略正确，那么贪心算法往往是易于描述和实现的。本节给出可以用贪心算法解决的若干经典问题的代码实现。

例 2-10 【贪心】装箱（Bin Packing, SWERC 2005, UVa 1149）

给定 N ($N \leq 10^5$) 个物品的重量 L_i ，背包的容量 M ，要求每个背包最多装两个物品。求至少要多少个背包，才能装下所有的物品。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <iostream>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
const int MAXN = 1e5 + 8;
int n, l, len[MAXN];
int solve() {
    cin >> n >> l;
    _for(i, 0, n) cin >> len[i];
    sort(len, len + n, greater<int>());
    int ans = 0, left = 0, right = n - 1;
    while (left <= right) {
        ans++, left++;
        if (left != right && len[left] + len[right] <= l) right--;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    _for(t, 0, T) {
        if (t) cout << endl;
        cout << solve() << endl;
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 8-1
注意：本题使用了双指针扫描法
同类题目：The Dragon of Loowater, UVa 11292
*/

```

例2-11 【贪心；选择不相交区间】今年暑假不AC（HDU 2037）

数轴上有 n 个开区间 (a_i, b_i) 。选择尽量多的区间，使得这些区间两两没有公共点。

【代码实现】

```

// 陈锋
#include <cstdio>
#include <algorithm>
using namespace std;

struct Seg {
    int a, b;
    bool operator < (const Seg &s) const {
        if (b != s.b) return b < s.b;
        return a > s.a;
    }
} A[100 + 4];

int main() {
    for (int n; scanf("%d", &n) == 1 && n; ) {
        for (int i = 0; i < n; i++) scanf("%d%d", &A[i].a, &A[i].b);
        sort(A, A + n);
        int ans = 1, cur_b = A[0].b;
        for (int i = 1; i < n; i++)
            if (A[i].a >= cur_b) ans++, cur_b = A[i].b;
        printf("%d\n", ans);
    }
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》8.4.2节选择不相交区间
同类题目：Gene Assembly, ACM/ICPC SouthAmerica 2001, ZOJ 1076
          Radar Installation, POJ 1328
*/

```

例2-12 【贪心；区间选点】高速公路（Highway, ACM/ICPC SEERC 2005, UVa 1615）

给定平面上有 n ($n \leq 10^5$) 个点和一个值 D ，要求在 x 轴上选出尽量少的点，使得对于给定的每个点，都有一个选出的点离它的欧几里得距离不超过 D 。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
using namespace std;
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

const double eps = 1e-4;
double dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}
typedef Point Segment;

const int MAXN = 1e5 + 8;
int L, D, N;
Point points[MAXN];
vector<Segment> segs;

// 圆[p,D]和x轴的两个交点
Segment getInterSeg(const Point& p) {
    double m = sqrt(D * D - p.y * p.y), x = p.x - m, y = p.x + m;
    if (dcmp(x) < 0) x = 0;
    if (dcmp(y - L) > 0) y = L;
    return Segment(x, y);
}

bool segcmp(const Segment& sl, const Segment& sr) {
    double yd = dcmp(sl.y - sr.y);
    return yd < 0 || (yd == 0 && dcmp(sl.x - sr.x) > 0);
}

void solve() {
    segs.clear();
    _for(i, 0, N) segs.push_back(getInterSeg(points[i]));
    sort(segs.begin(), segs.end(), segcmp);
    int ans = 1;
    double p = segs[0].y;
    for (size_t i = 1; i < segs.size(); i++) {
        const Segment &prev = segs[i - 1], &cur = segs[i];
        if (dcmp(cur.x - prev.x) < 0) continue;
        if (dcmp(cur.x - p) > 0) p = cur.y, ans++;
    }
    cout << ans << endl;
}

int main() {
    while (cin >> L >> D >> N) {
        _for(i, 0, N) cin >> points[i].x >> points[i].y;
        solve();
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——习题与解答》习题 8-11，《算法竞赛入门经典》8.4.5 节区间选点问题
*/

```

例2-13 【贪心；区间覆盖】 喷水装置 (Watering Grass, UVa 10382)

有一块草坪，长为 l ，宽为 w 。在其中心线的不同位置处装有 n ($1 \leq n \leq 10\,000$) 个点状的喷水装置。每个喷水装置 i 可将以它为中心，半径为 r_i 的圆形区域喷湿（见图2-2）。请选择尽量少的喷水装置，把整个草坪全部喷湿。输出需要打开的喷水装置数目的最小值。如果无解，输出-1。

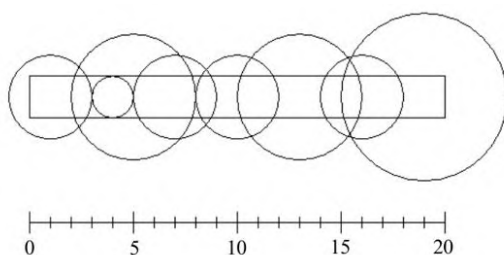


图2-2 喷水装置

【代码实现】

```

// 陈锋
#include <cstdio>
#include <cmath>
#include <vector>
#include <algorithm>
#include <functional>

using namespace std;
const int MAXN = 10240;
const double EPS = 1e-11;
double dcmp(double x) {
    if (fabs(x) < EPS) return 0;
    return x < 0 ? -1 : 1;
}

struct Seg {
    double a, b;
    inline bool operator< (const Seg& rhs) const {
        return a < rhs.a || (a == rhs.a && b < rhs.b);
    }
};

int solve(const vector<Seg> &segs, double len) {
    int cnt = 0;
    double start = 0.0, end = 0.0;
    for (size_t i = 0; i < segs.size(); i++) {
        start = end;
        if (dcmp(segs[i].a - start) > 0) return -1;
        cnt++;
        while (i < segs.size() && segs[i].a <= start) {
            if (dcmp(segs[i].b - end) > 0) end = segs[i].b;
            i++;
        }
        if (dcmp(end - len) > 0) break;
    }
    if (dcmp(end - len) < 0) cnt = -1;
    return cnt;
}

int main() {
    for (int n, l, w; scanf("%d%d%d", &n, &l, &w) == 3; ) {
        double len = l, w2 = w / 2.0;
        vector<Seg> segs;
        for (int i = 0, p, r; i < n; i++) {
            scanf("%d%d", &p, &r);
            if (r * 2 <= w) continue;
            double xw = sqrt((double)r * r - w2 * w2), a = p - xw;
            if (dcmp(a - len) > 0) continue;
            if (dcmp(a) <= 0) a = 0;
            segs.push_back({a, p + xw});
        }
        sort(segs.begin(), segs.end());
        printf("%d\n", solve(segs, len));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》8.4.2节区间覆盖问题
注意：本题使用 dcmp 对浮点数做比较，避免了浮点误差
同类题目：Cleaning Shifts, POJ 2376
*/

```

例2-14 【贪心法求最优排列】 突击战 (Commando War, UVa 11729)

你有 n ($1 \leq n \leq 1000$) 个部下，每个部下需要完成一项任务。第 i 个部下需要你花费 B_i 分钟交待任务，然后他会立刻独立、无间断地执行 J_i 分钟后完成任务 ($1 \leq B_i \leq 10\,000$, $1 \leq J_i \leq 10\,000$)。你需要选择交待任务的顺序，使得所有任务能尽早地执行完毕（即最后一个执行完的任务应尽早结束），输出所有任务完成的最短时间。注意，不能同时给两个部下交待任务，但部下们可以同时执行他们各自的任务。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

struct Job {
    int j, b;
    bool operator<(const Job& x) const {
        return j > x.j; // 运算符重载，不要忘记 const 修饰符
    }
};

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    for (int n, b, j, kase = 1; cin >> n && n; kase++) {
        vector<Job> v(n);
        for (int i = 0; i < n; i++) cin >> v[i].b >> v[i].j;
        sort(v.begin(), v.end()); // 使用 Job 类的 < 运算符排序
        int ans = 0;
        for (int i = 0, s = 0; i < n; i++) {
            s += v[i].b; // 当前任务的开始执行时间
            ans = max(ans, s + v[i].j); // 任务执行完毕时的最晚时间
        }
        printf("Case %d: %d\n", kase, ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 1.1 节例题 2
同类题目：Shoemaker's Problem, UVa 10026
          Schedule, HDU 6180
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表2-2所示。

表2-2 贪心算法例题归纳